Do We Need an LLM to Plan Retrieval? Lightweight Budget Controllers for SimpleMem

Joelle Barhoun
Ali Kheireddine
Ali Al Mokdad
Research Master's in Data Science
Lebanese University
October 21, 2026

Abstract

SimpleMem [1] gives LLM agents a compressed long-term memory, but its intent-aware retrieval planner makes an LLM call for every query and the surrounding pipeline spends further tokens on reflection. We ask whether this planning can be replaced by a lightweight controller. We implement two small decision models (LAYA and Jev) that predict four retrieval signals from the question, three budgeting strategies (complexity, retrieval, combined) that turn these signals into top-k values, and a fixed-k control. On one LoCoMo conversation (199 questions), the controllers reduce the average number of tokens per query by 76.8–86.3% relative to our reproduced baseline (8,444 tokens/query), while macro-F1 drops by 2.05 to 5.55 points from the baseline's 41.25. Jev with the complexity strategy offers the best compromise (39.2 macro-F1 with 77.1% fewer tokens). The fixed-k control reaches the lowest cost overall (k=3, 88.9% fewer tokens), and a static k=11 reaches 39.8 macro-F1 at the same cost as the best controller, so we find no evidence that adaptive depth outperforms a well-chosen constant. The one category where a controller clearly helps over a fixed budget is multi-hop questions.

List of Tables

1 SimpleMem F1 on LoCoMo by question type (published) . . . 2
2 Our baseline versus the SimpleMem paper . . . 5
3 F1, macro-F1 and token cost of the baseline and all variants . . . 5
4 BLEU-1 per question category for all variants . 6
5 Change in macro-F1 and token reduction versus our baseline . . . 6

List of Figures

1 Overview of the pipeline . . . 3

1 The Baseline Paper

1.1 Problem

Large Language Model (LLM) agents are constrained by fixed context windows. This is particularly relevant when agents engage in long-context and multi-turn interactions. For reliable long-term interaction, LLM agents require robust memory systems. Liu et al. [1] study how an LLM agent can maintain such a useful memory. Many existing systems maintain the full interaction history which introduces substantial redundant information. This redundancy affects memory retrieval negatively. On the other hand, some frameworks rely on iterative reasoning procedures to filter out the noise, resulting in substantial computational cost and increased latency and token usage. Neither paradigm achieves efficient allocation of memory and computation resources.

1.2 Solution

SimpleMem is a framework with a three-stage pipeline:

1. **Semantic Structured Compression:** the important idea is that a memory unit should contain enough context to be relevant without depending on the surrounding dialogue. In this stage, a window slides across the conversation and retrieves relevant information as judged by an LLM, and then references to people and times are resolved by replacing pronouns with names and relative dates with absolute dates.

2. **Online Semantic Synthesis:** Related facts within the session are integrated so that repetitive or fragmented details are merged into coherent memories rather than increasing storage redundancy. Entries are indexed in three representations: semantic (dense embeddings), lexical (keyword and terms), and symbolic (metadata on entities, persons, dates, and locations).

3. **Intent-Aware Retrieval Planning:** Different questions may require a different number and types of evidence. When a user asks a question, SimpleMem analyzes the information needs of the query. A planner LLM reasons about the information requirements of the query to produce a retrieval plan containing a query for the semantic index, a query for the lexical index, a query for the symbolic index, and the retrieval depth. The search is then performed and the retrieved memories are used as compact context for the answer.

1.3 Results

In the published LoCoMo evaluation using GPT-4.1-mini, SimpleMem reports 43.24% average F1, compared with

Table 1: SimpleMem F1 scores on LoCoMo by question type.

Question type	F1 (%)
Multi-hop	43.46
Temporal	58.62
Open-domain	19.76
Single-hop	51.12
Average	43.24

34.20% for Mem0, corresponding to a 26.4% relative increase. The paper reports 531 tokens/query for SimpleMem, compared with approximately 973 tokens/query for Mem0 and 16,910 tokens/query when sending the full context, and highlights lower retrieval cost and strong efficiency [1]. Table 1 summarizes the paper's reported F1 score for each LoCoMo question type.

2 Our Extension

2.1 Motivation

The intent-aware planner adds an LLM call to every query, which costs tokens and latency, and the planner may allocate retrieval depth inconsistently. We hypothesized that (a) a decision model can perform as a cheap predictor of query difficulty, setting retrieval depth just as well as an LLM planner, and (b) removing the reflection step would save the most tokens while affecting performance only slightly. The implemented scope includes two controllers (LAYA and Jev), three independently selectable budgeting strategies, and an inexpensive fixed-k run for comparison.

2.2 Method

Controller signals. Figure 1 gives an overview of the pipeline. The question is passed unchanged to a controller. The controller generates four signals (semantic, lexical, symbolic, and multi-hop) representing the predicted usefulness of the first three search approaches, and the likely need to combine memories.

Budgeting and retrieval. A budget policy converts those signals into requested top-k values. The retriever then runs semantic similarity search, keyword/full-text search, and metadata-based structured search. Results are interleaved by rank, deduplicated using IDs (`entry_id`), and truncated to at most 20 distinct memories before the existing answer generator runs.

LAYA controller. The controller is implemented in the Python file `extension/laya/laya_controller.py`, which configures a LAYA router (`laya.Router`) with four `noul` questions about retrieval usefulness and multi-hop requirements. The returned numbers are used as signals for the budgeting strategies explained below. The file `extension/laya/laya_retriever.py` implements extraction of query keywords and optional metadata filters, search execution, error diagnostics, and context merging.

Jev controller. The controller is implemented in the Python file `extension/jev/jev_controller.py`, which uses `TypeSafeClient.system_one` with model `jev-1.13.0` and goes through the same process as the LAYA controller. The file `extension/jev/jev_retriever.py` is analogous to `extension/laya/laya_retriever.py`.

Budgeting strategies. Let the query representation be

$$q = \big(s,\, l,\, y,\, h\big),$$

where s, l, y, and h denote the semantic, lexical, symbolic, and multi-hop signals, respectively. The complexity score is then defined as

$$C(q) = \min\{1,\, \max\{0,\, 0.2\,s + 0.2\,l + 0.2\,y + 0.4\,h\}\}.$$

The configured bounds are $k_{\min} = 3$ and $k_{\max} = 20$.

- **Complexity:** We request k memories from each applicable layer, where

  $$k = 3 + \mathrm{round}\,(17 \cdot C(q)).$$

  The final output is still capped at $k_{\max} = 20$ memories.

- **Retrieval:** Independently choose

  $$\begin{aligned} k_{\mathrm{s}} &= 3 + \mathrm{round}(17 \cdot s), \quad s \in [0,1], \\ k_{\mathrm{l}} &= 3 + \mathrm{round}(17 \cdot l), \quad l \in [0,1], \\ k_{\mathrm{y}} &= 3 + \mathrm{round}(17 \cdot y), \quad y \in [0,1], \end{aligned}$$

  where the resulting values k_{s}, k_{l}, and k_{y} determine the retrieval depth for the semantic, lexical, and symbolic searches, respectively.

- **Combined:** Determine one total request budget

  $$B = 3 + \mathrm{round}\,(17 \cdot C(q)),$$

  then distribute B proportionally among available semantic, lexical, and symbolic channels using largest-remainder rounding. When no valid symbolic filters exist, only semantic and lexical receive a share.

Filters and keywords. Person/entity/location filters must appear in the question and match values already present in stored memories. Year (YYYY) or month-year (Month YYYY) expressions are transformed into timestamp ranges. Therefore, structured retrieval is not automatically executed for every question. The keyword query is obtained deterministically by tokenization and removing stop words.

Fixed-k control. This control uses the same retrieval channels, rules, and bounds as above, but uses a constant k for each applicable channel. We test this on $k \in \{3, 7, 11, 15\}$. Its purpose is to provide the grounds to check whether controller budgeting is better than a simple static budget.

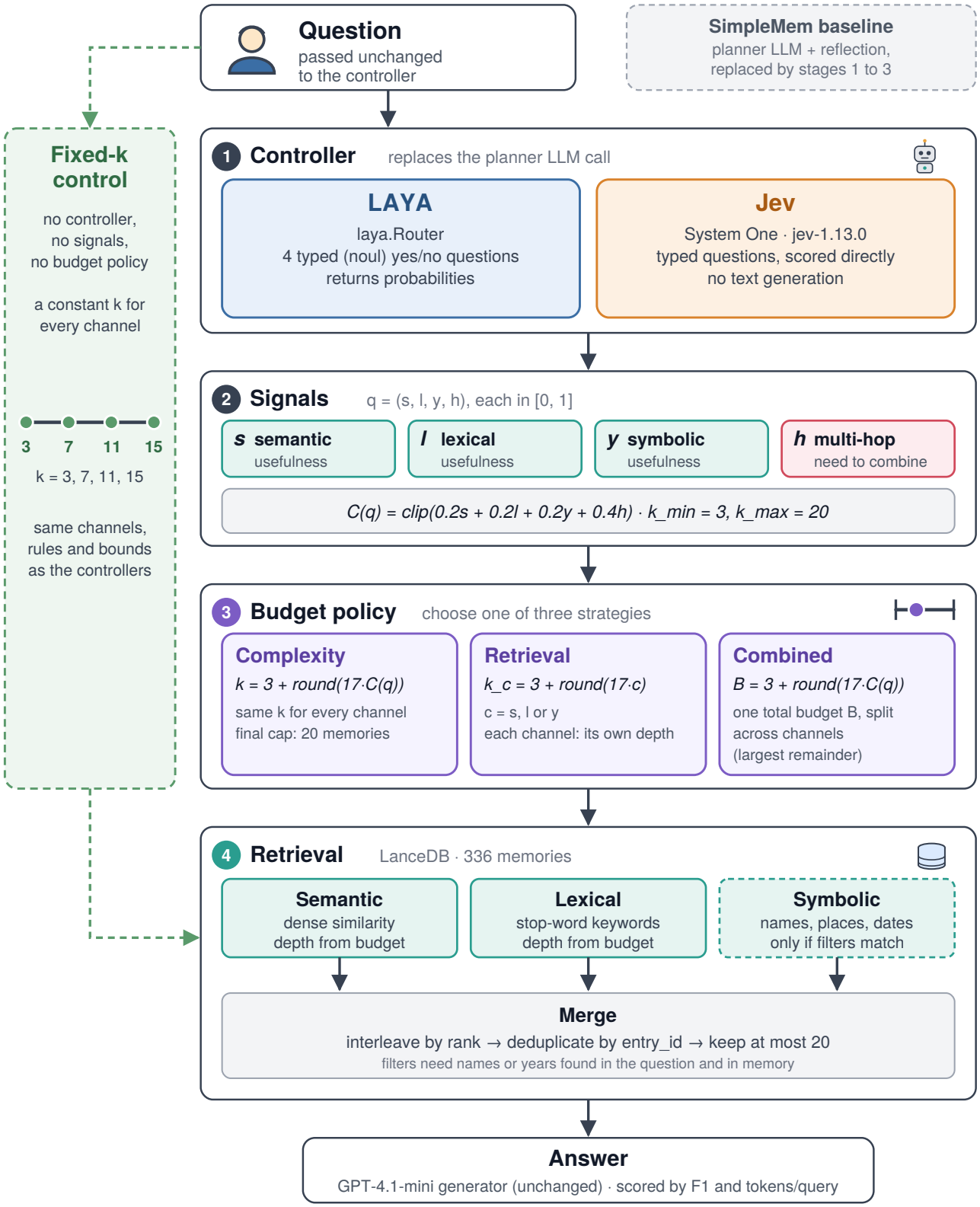


Figure 1: Overview of the pipeline: the question is passed to a controller, which produces the four signals; a budget policy converts them into top-k values for the semantic, lexical, and symbolic searches; and the merged memories are given to the answer generator.

2.3 Implementation notes

Code. The evaluated code is written in Python and retains the original SimpleMem components for memory representation, LanceDB storage, dense embeddings, and GPT-4.1-mini answer generation. All necessary packages are in `requirements.txt`. The Jev code additionally imports `typesafe_SDK` and expects a `TYPESAFE_API_KEY`.
Configuration. The Python file `config.py.example` must be copied to `config.py` and filled with your API key and selected model name.
Evaluation data. The common evaluation is one LoCoMo conversation (`conv-26`) with 199 question-answer pairs, derived from 19 sessions and 419 dialogue turns. The same LanceDB with 336 memories built by the baseline is reused for comparison, and each extension runner operates on a temporary database copy to avoid modifying the source memories. The obtained results are saved in the JSON file format. The saved questions include 32 multi-hop (category 1), 37 temporal (category 2), 13 open-domain (category 3), 70 single-hop (category 4), and 47 adversarial (category 5) items. [2]
Usage. Example invocations from the repository root, once the dataset, memory database, environment and dependencies are configured:

```
python extension/jev/run_jevmem_locomo.py
    --preflight-only
python extension/jev/run_jevmem_locomo.py
    --strategy retrieval
python extension/laya/run_layamem_locomo.py
    --strategy retrieval
    --dataset PATH_TO_LOCOMO_JSON
    --db-path PATH_TO_LANCEDB
python extension/fixed_k/run_fixed_k_locomo.py
    --k 3 7 11 15
```

Every `run_*_locomo.py` script also has the option to specify the number of LoCoMo samples. We ran our experiments with `python extension/*/run_*_locomo.py --num-samples 1`

Outputs. The result outputs are saved in the following JSON files: `baseline/baseline_result.json`, `extension/laya/layamem_*_results.json`, `extension/jev/jevmem_*_results.json`, and `extension/fixed_k/fixedk_results/fixedk_k*_results.json`.

3 Results

3.1 Reproduction of the baseline

Token cost. The SimpleMem paper reports an average of 531 tokens per query. When we cloned the authors' released code and ran it unmodified, our baseline measured 8,444 tokens per query, roughly 16× higher. The paper does not specify how its token metric was computed, so we cannot determine whether the gap comes from a different counting method or from a difference in setup. Therefore, all comparisons in our report are made against our own reproduced baseline of 8,444 tokens per query, with every variant (fixed-k, LAYA, Jev) measured using the same counting method. Our token cost results should therefore be read as relative to our reproduction, not to the paper's published number.
Comparison with the paper. Since we only ran the baseline code on the first LoCoMo sample (Sample 0), we provide a comparison with the SimpleMem paper in Table 2.

3.2 Extension versus baseline

Table 3 reports the F1 for each question category, the macro-F1 over categories 1–4 (multi-hop, temporal, open-domain, and single-hop), the average number of tokens per query, and the token reduction for our reproduced baseline and all ten variants, and Table 4 reports the BLEU-1 per category. All variants are evaluated on the same 199 questions and the same memory database as the baseline, which reaches 41.25% macro-F1 at 8,444 tokens per query. Table 5 restates the macro-F1 and token columns as changes relative to our baseline.
Controllers versus the baseline. Replacing the planner with a controller cuts the cost sharply. Jev with the retrieval and complexity strategies uses 1,957.6 and 1,934.1 tokens per query (76.8% and 77.1% fewer than the baseline), Jev with the combined strategy uses 1,158.7 (86.3% fewer), and the three LAYA strategies use 1,215.7–1,215.9 (85.6% fewer). The price is a drop in macro-F1 of 2.05 and 2.25 points for Jev with complexity and retrieval, 3.75 points for Jev with combined, and 3.95 (complexity), 4.25 (combined) and 5.55 points (retrieval) for LAYA. In other words, the best controllers stay within about 2 macro-F1 points of the baseline for less than a quarter of its tokens, and even the least accurate controller (LAYA with retrieval) remains within 5.55 points at about 14% of the cost.
Jev versus LAYA. Jev reaches a higher macro-F1 than LAYA under every strategy: 39.0 versus 35.7 with retrieval, 39.2 versus 37.3 with complexity, and 37.5 versus 37.0 with combined. LAYA is cheaper than Jev with the retrieval and complexity strategies (about 1,216 versus 1,934–1,958 tokens per query), but Jev with the combined strategy is both cheaper (1,158.7 tokens) and more accurate than all three LAYA runs.
Question categories. The baseline reaches 39.8 F1 on multi-hop, 58.4 on temporal, 14.1 on open-domain, and 52.6 on single-hop questions. Every variant scores below it on multi-hop, temporal, and single-hop questions, and the macro-F1 loss comes mainly from the first two. Multi-hop questions show the widest spread: the loss ranges from 2.9 points (Jev with retrieval) to 16.6 points (fixed k=3), and only Jev with the retrieval and complexity strategies stay within 4 points of the baseline (36.9 and 36.1). On temporal questions the variants lose between 2.5 and 5.9 points, and on single-hop questions between 0.8 and 4.4

Table 2: Comparison between our baseline evaluation and the SimpleMem paper.

Metric	**SimpleMem Paper**	**Our Baseline**
Benchmark samples	Entire LoCoMo-10 benchmark	Sample 0
Evaluation size	1986 questions	199 questions
Average F1 (macro, %)	43.24	41.25
Token cost	531 tk/q	8,444 tk/q

Table 3: Comparison of our reproduced baseline and all variants. F1 (%) per question category (1–4), macro-F1 (%) over categories 1–4, average tokens per query, and token reduction relative to our baseline.

Method	Multi-hop	Temporal	Open-domain	Single-hop	Macro F1	Tokens/Q	Token red. (%)
SimpleMem (our baseline)	39.8	58.4	14.1	52.6	41.3	8,444	–
Jev (retrieval)	36.9	52.8	14.9	51.5	39.0	1,957.6	76.8
Jev (complexity)	36.1	54.8	16.6	49.3	39.2	1,934.1	77.1
Jev (combined)	25.0	54.8	19.7	50.4	37.5	1,158.7	86.3
LAYA (retrieval)	27.6	52.5	14.7	48.2	35.7	1,215.8	85.6
LAYA (complexity)	27.5	54.5	17.4	49.7	37.3	1,215.7	85.6
LAYA (combined)	28.3	54.0	16.9	48.8	37.0	1,215.9	85.6
Fixed-k (k=3)	23.2	55.6	17.4	49.1	36.3	938.4	88.9
Fixed-k (k=7)	30.9	52.7	15.4	51.8	37.7	1,736.1	79.4
Fixed-k (k=11)	33.0	55.9	20.7	49.4	39.8	1,931.5	77.1
Fixed-k (k=15)	32.4	55.4	16.6	48.6	38.3	1,956.7	76.8

points. Open-domain questions are the exception: every variant scores above the baseline (14.7 to 20.7), but this category has only 13 questions and we do not interpret the difference.

BLEU-1. Table 4 gives the BLEU-1 of each variant per category next to the baseline (36.4 multi-hop, 47.6 temporal, 17.1 open-domain, 45.5 single-hop). It follows F1 where the baseline is strongest: every variant is below the baseline on multi-hop questions (by 3.1 to 18.5 points) and on temporal questions (by 2.4 to 6.0 points), and Jev with retrieval and complexity are again the closest on multi-hop (33.3 and 32.9). On single-hop questions BLEU-1 is more favorable than F1: four variants reach or exceed the baseline (Jev with combined 46.7, Jev with retrieval 46.5, fixed k=7 46.2, and LAYA with complexity 45.8), whereas none does in F1. On open-domain questions five variants exceed the baseline, up to 21.7 for fixed k=11, and the remaining five are at most 2.3 points below it.

Fixed k as a reference. The fixed-k runs show the cost floor of the pipeline. The lowest cost of all variants is obtained with k=3 (938.4 tokens per query, 88.9% fewer than the baseline), at a macro-F1 of 36.3, which is the second-largest loss in Table 5 (-4.95). Raising k to 11 recovers a macro-F1 of 39.8 (-1.45 relative to the baseline) at 1,931.5 tokens per query (77.1% fewer), while at k=15 the cost barely changes (1,956.7) and macro-F1 falls to 38.3. The gain from depth comes almost entirely from multi-hop questions (23.2 F1 at k=3 versus 33.0 at k=11); temporal and single-hop F1 stay within a 3.2-point band across all four values of k (52.7–55.9 and 48.6–51.8), and open-domain F1 fluctuates between 15.4 and 20.7.

3.3 Honest assessment

Controllers versus the baseline. In terms of cost the controllers clearly improve on the baseline, with 76.8–86.3% fewer tokens per query. In terms of accuracy they pay a price that depends on the controller: Jev stays within 2.05–3.75 macro-F1 points of the baseline and LAYA within 3.95–5.55. The smaller of these gaps should be read with caution given the size of our evaluation (see Section 5). Per category, the loss relative to the baseline comes mainly from multi-hop and temporal questions, while every variant scores above the baseline on open-domain questions, a category too small (13 questions) to interpret.

Hypothesis (a) is partly supported. Jev with the complexity or retrieval strategy comes within 2.05 and 2.25 macro-F1 points of the LLM planner, while LAYA lands 3.95 to 5.55 points below it. Jev with the combined strategy is both cheaper and more accurate than every LAYA run. The fixed-k control gives a reference point for how much of this accuracy is due to adaptive depth: a static k=11 scores 39.8, above every controller, at essentially the same cost as the best one (1,931.5 versus 1,934.1 tokens/query for Jev with complexity). In the low-budget range, Jev (combined) and LAYA (complexity) sit about 0.8 and 0.5 points above the straight line joining k=3 and k=7 in the tokens-versus-F1 plane. We do not consider margins this small to be reliable, so we cannot credit the controllers' accuracy to adaptive depth alone.

Where controllers help: multi-hop questions. The fixed-k runs show that depth matters for multi-hop questions but hardly for temporal or single-hop ones. On

Table 4: Comparison of our reproduced baseline and all variants on BLEU-1 (%) per question category (1–4).

Method	Multi-hop	Temporal	Open-domain	Single-hop
SimpleMem (our baseline)	36.4	47.6	17.1	45.5
Jev (retrieval)	33.3	41.6	17.8	46.5
Jev (complexity)	32.9	43.5	19.6	44.7
Jev (combined)	18.2	44.3	17.0	46.7
LAYA (retrieval)	22.5	42.4	14.8	44.1
LAYA (complexity)	22.4	43.3	16.2	45.8
LAYA (combined)	23.2	43.1	16.2	44.8
Fixed-k (k=3)	17.9	44.8	16.5	45.3
Fixed-k (k=7)	26.4	42.0	17.8	46.2
Fixed-k (k=11)	29.1	45.2	21.7	45.1
Fixed-k (k=15)	28.7	45.1	18.3	43.8

Table 5: Change in macro-F1 relative to our reproduced baseline (41.25), in points, and token reduction relative to our baseline ($1 - \text{tokens}/8{,}444$).

Method	ΔF1 vs. our baseline	Token red. (%)
Jev (retrieval)	−2.25	76.8
Jev (complexity)	−2.05	77.1
Jev (combined)	−3.75	86.3
LAYA (retrieval)	−5.55	85.6
LAYA (complexity)	−3.95	85.6
LAYA (combined)	−4.25	85.6
Fixed-k (k=3)	−4.95	88.9
Fixed-k (k=7)	−3.55	79.4
Fixed-k (k=11)	−1.45	77.1
Fixed-k (k=15)	−2.95	76.8

multi-hop questions, Jev with the retrieval and complexity strategies reaches 36.9 and 36.1 F1, which is 3.9 and 3.1 points above the best fixed-k run at the same cost and within 2.9 and 3.7 points of the baseline (39.8). This is consistent with the multi-hop signal of Jev being informative, which is also the signal weighted most heavily in $C(q)$. LAYA does not show this effect: its multi-hop F1 (27.5–28.3) fall between those of k=3 and k=7 despite a cost of about 1,216 tokens/query, so its signals do not appear to identify the questions that need more evidence. We did not inspect the signal distributions directly, so this explanation remains a hypothesis.

Budget strategies. The complexity strategy ranks first for both controllers, although its margin over the retrieval strategy for Jev is only 0.2 points (39.2 versus 39.0). The combined strategy is the cheapest for Jev: splitting one total budget B across channels cuts tokens by 40.1% relative to Jev with complexity (1,158.7 versus 1,934.1), but multi-hop F1 drops from 36.1 to 25.0, close to the k=3 level, and macro-F1 drops by 1.7 points. A single total budget is therefore too tight for multi-hop questions, whose evidence is spread across channels. For LAYA, the three strategies cost the same within 0.2 tokens (1,215.7–1,215.9) yet differ by up to 1.6 macro-F1 points. Such identical costs suggest that the LAYA signals vary little from question to question, so that every strategy requests similar depths regardless of the question. We did not verify this.

Hypothesis (b) is only weakly tested. The savings we hoped for are large (76.8–86.3% for the controllers), and for the best configurations the loss is modest (2.05 to 2.25 macro-F1 points for Jev), which is in line with the hypothesis. However, the fixed-k controls obtain similar savings (76.8–88.9%) without any controller, so the savings come largely from the shared design (a bounded context of at most 20 memories and a single retrieval pass) rather than from the controller itself. Cost also saturates quickly: k=11 and k=15 differ by only 25 tokens per query, consistent with the 20-memory cap and deduplication, and the larger k does not help (−1.5 macro-F1). Every variant differs from the baseline in more than one component, and our experiments contain no ablation that removes the reflection step alone, so we cannot attribute the token savings to it specifically.

Answer to the title question. A lightweight controller can replace the planner LLM at a modest cost in accuracy: the controllers keep macro-F1 within 2.05 to 5.55 points of our baseline (41.25) with 76.8–86.3% fewer tokens per query, and Jev with the complexity strategy is the best compromise (39.2 macro-F1, 77.1% fewer tokens). The baseline planner still obtains the highest macro-F1 over-

all, although a gap of this size on one conversation of 152 scored questions is not something we can establish as reliable. The lowest cost is reached by the fixed-k control with k=3 (88.9% fewer tokens), and a tuned static budget (k=11) matches or exceeds the controllers' macro-F1, so the controllers' advantage over a fixed budget lies in multi-hop questions rather than in overall accuracy.

4 Related Work

Memory for LLM agents. MemGPT [4] treats the context window as a limited resource and lets the LLM move information between the prompt and external storage through function calls, so retrieval is directed by the model itself. Mem0 [3] extracts, consolidates and retrieves salient facts from conversations, and also proposes a graph-based variant; it reports lower latency and token cost than full-context processing. A-Mem [5] organizes memories as linked notes in the spirit of the Zettelkasten method, and lets new memories update existing ones. SimpleMem [1] focuses on compressing interactions into self-contained, multi-view indexed units and on planning retrieval from the query. These systems differ mainly in how memory is written and organized; how deep to retrieve at query time is either fixed or decided by an LLM, which is the point we study.

Benchmark. LoCoMo [2] consists of very long-term conversations over many sessions (up to 32 in the published version) with question answering, event summarization and multimodal dialogue generation tasks. Its authors found that long-context models and retrieval augmentation help but remain well below human performance. We use its question-answering task and its category structure (multi-hop, temporal, open-domain, single-hop, adversarial).

Adaptive retrieval. Adaptive-RAG [6] trains a small language model to predict query complexity and uses it to select between no retrieval, single-step retrieval and multi-step retrieval, with training labels obtained from model outcomes and dataset biases. Our controllers play a similar role, but they set the retrieval depth per channel rather than choose a strategy, and we used them as provided, without task-specific training. Self-RAG [7] trains a language model to decide when to retrieve and to critique its own outputs with reflection tokens, motivated by the observation that retrieving a fixed number of passages regardless of need can hurt. Our finding that k=15 does not improve over k=11 points in the same direction, although our setting is a memory store rather than open-domain QA.

Decision models as controllers. LAYA [8] is a router that answers typed yes/no questions (`noul`) over a text input and returns the probability of a positive answer. Jev [9] is a model from the System One family: it receives a state and a set of typed questions and returns structured answers directly, without generating text, evaluating the questions in parallel. Because both return scores, a retrieval plan can be derived without autoregressive generation, which is what makes them candidates for replacing a planner LLM.

5 Limitations

Sample size. All results come from one conversation (`conv-26`) with 199 questions, of which 152 (categories 1–4) enter the macro-F1. Category 3 contains 13 questions, so a single question can move its F1 by several points. We report one run per configuration without confidence intervals, so differences of one or two macro-F1 points, including the margins between the best controller and the best fixed-k run, should not be read as established. The paper's own figures are computed over the full LoCoMo-10 benchmark (1,986 questions), which we did not run.

Token metric. Our baseline uses 8,444 tokens/query against the 531 published, and we could not explain the gap. Token figures are therefore only meaningful relative to our baseline, and for this reason we compare all variants only against our reproduced baseline and not against the published results. We report tokens only; wall-clock latency was not measured, and Table 3 gives a single tokens/query figure per method without separating the cost of the controller call itself.

Controllers as black boxes. We did not inspect the distributions of the four signals. The explanations offered above for LAYA's near-identical costs and weak multi-hop performance are untested. The weights in $C(q)$ (0.2, 0.2, 0.2, 0.4), the bounds $k_{\min}$=3 and $k_{\max}$=20, and the signal-to-k mapping were fixed by hand and not tuned, and the fixed-k grid has only four points. Jev is a hosted service; we pinned `jev-1.13.0`, but reproducing our runs depends on its continued availability.

Missing ablation. We did not isolate the reflection step from the planner, so hypothesis (b) cannot be separated from hypothesis (a) in our results.

Fixed memory and metrics. The memory database (336 memories) was built once by the baseline and reused, so all variants are conditioned on memories produced under the original compression and indexing, and the answer generator is fixed to GPT-4.1-mini. F1 and BLEU-1 measure lexical overlap and penalize correct answers phrased differently from the reference; no LLM-based judging was used. The adversarial category (47 questions) is excluded from the macro-F1.

Next steps. The most informative follow-ups follow directly from these points: run all ten LoCoMo conversations with bootstrap confidence intervals, log and analyze the controller signals, ablate reflection and the planner separately, and train a small classifier on labels indicating the smallest k that answers each question correctly, in the manner of Adaptive-RAG [6].

References

[1] J. Liu, Y. Su, P. Xia, S. Han, Z. Zheng, C. Xie, M. Ding, H. Yao. SimpleMem: Efficient Lifelong Memory for LLM Agents. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, PMLR 306, 2026. arXiv:2601.02553.

[2] A. Maharana, D.-H. Lee, S. Tulyakov, M. Bansal, F. Barbieri, Y. Fang. Evaluating Very Long-Term Conversational Memory of LLM Agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024. arXiv:2402.17753.

[3] Mem0 research team. Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory. arXiv:2504.19413, 2025.

[4] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, J. E. Gonzalez. MemGPT: Towards LLMs as Operating Systems. arXiv:2310.08560, 2023.

[5] W. Xu, Z. Liang, K. Mei, H. Gao, J. Tan, Y. Zhang. A-Mem: Agentic Memory for LLM Agents. In *Advances in Neural Information Processing Systems 38 (NeurIPS)*, 2025.

[6] S. Jeong, J. Baek, S. Cho, S. J. Hwang, J. Park. Adaptive-RAG: Learning to Adapt Retrieval-Augmented Large Language Models through Question Complexity. In *Proceedings of NAACL*, 2024. arXiv:2403.14403.

[7] A. Asai, Z. Wu, Y. Wang, A. Sil, H. Hajishirzi. Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.11511.

[8] Laya: typed-question router (`noul` questions). Python package `laya` and Hugging Face model documentation, accessed October 2026.

[9] TypeSafe AI. System One API and the Jev models (`jev-1.13.0`). Documentation at docs.typesafe.ai, accessed October 2026.